DKL /
data-catalog-api

Code | Issues | Pull requests 14 | Actions | Projects | Wiki | Security

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. Learn more about diff comparisons here.

base: develop ← compare: DATAUSER-5787-Endpoint-for-...

✓ **Able to merge.** These branches can be automatically merged.

Reviewers

Suggestions

A466449 Request

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use Closing keywords in the description to automatically close issues

2 commits | 22 files changed | 1 contributor

Commits on Apr 6, 2026

feat(attributes): Azure metadata Dyanamic Attributes

viknesh-vallinayagam committed 12 minutes ago

7713de0

feat(attribute): Enhancement of the endpoint structure to make it ava... ··· 5a9e6fa

viknesh-vallinayagam committed 10 minutes ago

Showing 22 changed files with 2,487 additions and 8 deletions. Split Unified

12

src/main/java/com/socgen/bigdata/catalog/exceptions/RestResponseEntityExceptionHandler.java

```
30  30          HashMap<String, List<String>> errors = new HashMap<>();
31  31          errors.put("domain", Collections.singletonList("insufficientRights"));
32  32          InvalidInputBody invalidInputBody = new
    InvalidInputBody("insufficientRights", errors);
33      -
34  33          return ResponseEntity.status(403).body(invalidInputBody);
35  34      }
36      - }
    35  +
    36  +   @ExceptionHandler(IllegalArgumentException.class)
    37  +   public ResponseEntity<InvalidInputBody>
        illegalArgument(IllegalArgumentException exception, WebRequest request) {
    38  +       InvalidInputBody body = new InvalidInputBody(
    39  +           exception.getMessage(),
    40  +           java.util.Map.of("error",
        Collections.singletonList(exception.getMessage()))
    41  +       );
    42  +       return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    43  +   }
    44  + }
```

49

...n/java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeDefinitionRequestDto.ja...

```
...  ...  @@ -0,0 +1,49 @@
     1  + package com.socgen.bigdata.catalog.models.dto.attribute;
     2  +
     3  + import com.fasterxml.jackson.annotation.JsonProperty;
     4  + import java.util.List;
     5  + import javax.validation.Valid;
     6  + import javax.validation.constraints.NotBlank;
     7  + import javax.validation.constraints.NotEmpty;
     8  + import lombok.AllArgsConstructor;
     9  + import lombok.Builder;
    10  + import lombok.Data;
    11  + import lombok.NoArgsConstructor;
    12  +
    13  + @Data
    14  + @Builder
```

```
+ @NoArgsConstructor
+ @AllArgsConstructor
+ public class AttributeDefinitionRequestDto {
+
+     @NotBlank(message = "category must not be blank")
+     @JsonProperty("category")
+     private String category;
+
+     @NotEmpty(message = "attributes list must not be empty")
+     @Valid
+     @JsonProperty("attributes")
+     private List<AttributeItemDto> attributes;
+
+     @Data
+     @Builder
+     @NoArgsConstructor
+     @AllArgsConstructor
+     public static class AttributeItemDto {
+
+         @NotBlank(message = "name must not be blank")
+         @JsonProperty("name")
+         private String name;
+
+         @NotBlank(message = "technicalName must not be blank")
+         @JsonProperty("technicalName")
+         private String technicalName;
+
+         @NotBlank(message = "dataType must not be blank")
+         @JsonProperty("dataType")
+         private String dataType;
+
+         @JsonProperty("readOnly")
+         private Boolean readOnly;
+     }
+ }
```

40

.../java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeDefinitionResponseDto.ja…

```
@@ -0,0 +1,40 @@
+ package com.socgen.bigdata.catalog.models.dto.attribute;
+
+ import com.fasterxml.jackson.annotation.JsonProperty;
+ import java.util.List;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Data;
+ import lombok.NoArgsConstructor;
+
```

```
+ @Data
+ @Builder
+ @NoArgsConstructor
+ @AllArgsConstructor
+ public class AttributeDefinitionResponseDto {
+
+     @JsonProperty("objectType")
+     private String objectType;
+
+     @JsonProperty("category")
+     private String category;
+
+     @JsonProperty("createdOrUpdated")
+     private List<CreatedAttributeDto> createdOrUpdated;
+
+     @Data
+     @Builder
+     @NoArgsConstructor
+     @AllArgsConstructor
+     public static class CreatedAttributeDto {
+
+         @JsonProperty("attributeDefId")
+         private Long attributeDefId;
+
+         @JsonProperty("name")
+         private String name;
+
+         @JsonProperty("technicalName")
+         private String technicalName;
+     }
+ }
```

33

src/main/java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeUpdateRequestDto.ja…

```
@@ -0,0 +1,33 @@
+ package com.socgen.bigdata.catalog.models.dto.attribute;
+
+ import com.fasterxml.jackson.annotation.JsonProperty;
+ import javax.validation.constraints.NotBlank;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Data;
+ import lombok.NoArgsConstructor;
+
+ @Data
+ @Builder
+ @NoArgsConstructor
+ @AllArgsConstructor
```

```
+ public class AttributeUpdateRequestDto {
+
+     @NotBlank(message = "name must not be blank")
+     @JsonProperty("name")
+     private String name;
+
+     @NotBlank(message = "technicalName must not be blank")
+     @JsonProperty("technicalName")
+     private String technicalName;
+
+     @NotBlank(message = "dataType must not be blank")
+     @JsonProperty("dataType")
+     private String dataType;
+
+     @JsonProperty("category")
+     private String category;
+
+     @JsonProperty("readOnly")
+     private Boolean readOnly;
+ }
```

25

src/main/java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeValueRequestDto.java

```
@@ -0,0 +1,25 @@
+ package com.socgen.bigdata.catalog.models.dto.attribute;
+
+ import com.fasterxml.jackson.annotation.JsonProperty;
+ import java.util.Map;
+ import javax.validation.constraints.NotBlank;
+ import javax.validation.constraints.NotEmpty;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Data;
+ import lombok.NoArgsConstructor;
+
+ @Data
+ @Builder
+ @NoArgsConstructor
+ @AllArgsConstructor
+ public class AttributeValueRequestDto {
+
+     @NotBlank(message = "category must not be blank")
+     @JsonProperty("category")
+     private String category;
+
+     @NotEmpty(message = "values must not be empty")
+     @JsonProperty("values")
+     private Map<String, Object> values;
```

```
+ }
```

36

src/main/java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeValueResponseDto.ja…

```
@@ -0,0 +1,36 @@
+ package com.socgen.bigdata.catalog.models.dto.attribute;
+
+ import com.fasterxml.jackson.annotation.JsonProperty;
+ import java.util.Map;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Data;
+ import lombok.NoArgsConstructor;
+
+ /**
+  * Response DTO for POST /v2/api/object-types/{objectType}/{objectId}/attribute-values
+  *
+  * Returns ONLY the attribute values that were pushed in this request.
+  * The full stored map is not returned — only what was just upserted.
+  *
+  * values key = attribute_id as string, value = the pushed value.
+  * e.g. if you pushed {"numberOfFiles": 21}, response is {"1": 21}
+  */
+ @Data
+ @Builder
+ @NoArgsConstructor
+ @AllArgsConstructor
+ public class AttributeValueResponseDto {
+
+     @JsonProperty("objectType")
+     private String objectType;
+
+     @JsonProperty("objectId")
+     private Long objectId;
+
+     @JsonProperty("category")
+     private String category;
+
+     @JsonProperty("values")
+     private Map<String, Object> values;
+ }
```

77

src/main/java/com/socgen/bigdata/catalog/models/dto/attribute/AttributeViewResponseDto.java

```
@@ -0,0 +1,77 @@
```

```
+ package com.socgen.bigdata.catalog.models.dto.attribute;
+
+ import com.fasterxml.jackson.annotation.JsonProperty;
+ import java.util.List;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Data;
+ import lombok.NoArgsConstructor;
+
+ /**
+  * Response DTO for GET /v2/api/object-types/{objectType}/{objectId}/attribute-values
+  *
+  * Returns all attribute values grouped by category.
+  * Each attribute entry includes dataType and readOnly so the frontend
+  * can build JSON Schema and control editability per field.
+  */
+ @Data
+ @Builder
+ @NoArgsConstructor
+ @AllArgsConstructor
+ public class AttributeViewResponseDto {
+
+     @JsonProperty("objectType")
+     private String objectType;
+
+     @JsonProperty("objectId")
+     private Long objectId;
+
+     @JsonProperty("categories")
+     private List<CategoryDto> categories;
+
+     @Data
+     @Builder
+     @NoArgsConstructor
+     @AllArgsConstructor
+     public static class CategoryDto {
+
+         @JsonProperty("category")
+         private String category;
+
+         @JsonProperty("attributes")
+         private List<AttributeEntryDto> attributes;
+     }
+
+     @Data
+     @Builder
+     @NoArgsConstructor
+     @AllArgsConstructor
+     public static class AttributeEntryDto {
+
+         @JsonProperty("attributeId")
```

```
52  +         private Long attributeId;
53  +
54  +         @JsonProperty("name")
55  +         private String name;
56  +
57  +         @JsonProperty("technicalName")
58  +         private String technicalName;
59  +
60  +         /**
61  +          * JSON Schema type string for JSON Forms rendering.
62  +          * Values: "string" | "integer" | "number" | "boolean" | "array" |
    "object"
63  +          */
64  +         @JsonProperty("dataType")
65  +         private String dataType;
66  +
67  +         /**
68  +          * If true — field is rendered as readonly in JSON Forms UI.
69  +          * Driven by ATTRIBUTE.READ_ONLY column, not hardcoded on frontend.
70  +          */
71  +         @JsonProperty("readOnly")
72  +         private Boolean readOnly;
73  +
74  +         @JsonProperty("value")
75  +         private Object value;
76  +     }
77  + }
```

73

src/main/java/com/socgen/bigdata/catalog/models/jpa/attribute/Attribute.java

```
...  ...  @@ -0,0 +1,73 @@
1   + package com.socgen.bigdata.catalog.models.jpa.attribute;
2   +
3   + import java.io.Serial;
4   + import java.io.Serializable;
5   + import java.util.Date;
6   + import javax.persistence.Column;
7   + import javax.persistence.Entity;
8   + import javax.persistence.GeneratedValue;
9   + import javax.persistence.GenerationType;
10  + import javax.persistence.Id;
11  + import javax.persistence.Table;
12  + import lombok.AllArgsConstructor;
13  + import lombok.Builder;
14  + import lombok.Getter;
15  + import lombok.RequiredArgsConstructor;
16  + import lombok.Setter;
17  + import lombok.ToString;
```

```
18 + import org.hibernate.annotations.ColumnTransformer;
19 + import org.hibernate.annotations.DynamicInsert;
20 + import org.hibernate.annotations.DynamicUpdate;
21 + import org.hibernate.annotations.Generated;
22 + import org.hibernate.annotations.GenerationTime;
23 +
24 + @Getter
25 + @Setter
26 + @ToString
27 + @Builder
28 + @AllArgsConstructor
29 + @RequiredArgsConstructor
30 + @Entity
31 + @DynamicInsert
32 + @DynamicUpdate
33 + @Table(name = "ATTRIBUTE")
34 + public class Attribute implements Serializable {
35 +
36 +     @Serial
37 +     private static final long serialVersionUID = 1L;
38 +
39 +     @Id
40 +     @GeneratedValue(strategy = GenerationType.IDENTITY)
41 +     @Column(name = "ID")
42 +     private Long id;
43 +
44 +     @Column(name = "NAME", nullable = false)
45 +     private String name;
46 +
47 +     @Column(name = "TECHNICAL_NAME", nullable = false)
48 +     private String technicalName;
49 +
50 +     @Column(name = "DATA_TYPE", nullable = false)
51 +     private String dataType;
52 +
53 +     @Column(name = "ENTITY_TYPE", nullable = false)
54 +     private String objectType;
55 +
56 +     @Column(name = "CATEGORY", nullable = false)
57 +     private String category;
58 +
59 +     @Column(name = "READ_ONLY", nullable = false)
60 +     private Boolean readOnly;
61 +
62 +     @Column(name = "PROPERTIES", columnDefinition = "jsonb")
63 +     @ColumnTransformer(write = "?::jsonb")
64 +     private String properties;
65 +
66 +     @Generated(GenerationTime.INSERT)
67 +     @Column(name = "CREATED_AT")
68 +     private Date createdAt;
69 +
```

```
+     @Generated(GenerationTime.ALWAYS)
+     @Column(name = "UPDATED_AT")
+     private Date updatedAt;
+ }
```

33

src/main/java/com/socgen/bigdata/catalog/models/jpa/attribute/AttributeDataType.java

```
@@ -0,0 +1,33 @@
+ package com.socgen.bigdata.catalog.models.jpa.attribute;
+
+ public enum AttributeDataType {
+     STRING,
+     INTEGER,
+     FLOAT,
+     BOOLEAN;
+
+     public static AttributeDataType fromString(String value) {
+         if (value == null) {
+             throw new IllegalArgumentException("dataType must not be null.");
+         }
+         try {
+             return AttributeDataType.valueOf(value.toUpperCase());
+         } catch (IllegalArgumentException e) {
+             throw new IllegalArgumentException(
+                 String.format(
+                     "Invalid dataType '%s'. Allowed values: string, integer, float, boolean, array, object.",
+                     value
+                 )
+             );
+         }
+     }
+
+     public String toJsonSchemaType() {
+         return switch (this) {
+             case STRING  -> "string";
+             case INTEGER -> "integer";
+             case FLOAT   -> "number";
+             case BOOLEAN -> "boolean";
+         };
+     }
+ }
```

61

src/main/java/com/socgen/bigdata/catalog/models/jpa/attribute/AttributeValue.java

```
@@ -0,0 +1,61 @@
```

```
+ package com.socgen.bigdata.catalog.models.jpa.attribute;
+
+ import java.io.Serial;
+ import java.io.Serializable;
+ import java.util.Date;
+ import javax.persistence.Column;
+ import javax.persistence.Entity;
+ import javax.persistence.GeneratedValue;
+ import javax.persistence.GenerationType;
+ import javax.persistence.Id;
+ import javax.persistence.Table;
+ import lombok.AllArgsConstructor;
+ import lombok.Builder;
+ import lombok.Getter;
+ import lombok.RequiredArgsConstructor;
+ import lombok.Setter;
+ import lombok.ToString;
+ import org.hibernate.annotations.ColumnTransformer;
+ import org.hibernate.annotations.DynamicInsert;
+ import org.hibernate.annotations.DynamicUpdate;
+ import org.hibernate.annotations.Generated;
+ import org.hibernate.annotations.GenerationTime;
+
+ @Getter
+ @Setter
+ @ToString
+ @Builder
+ @AllArgsConstructor
+ @RequiredArgsConstructor
+ @Entity
+ @DynamicInsert
+ @DynamicUpdate
+ @Table(name = "ATTRIBUTE_VALUE")
+ public class AttributeValue implements Serializable {
+
+     @Serial
+     private static final long serialVersionUID = 2L;
+
+     @Id
+     @GeneratedValue(strategy = GenerationType.IDENTITY)
+     @Column(name = "ID")
+     private Long id;
+
+     @Column(name = "ENTITY_TYPE", nullable = false)
+     private String objectType;
+
+     @Column(name = "ENTITY_ID", nullable = false)
+     private Long objectId;
+
+     @Column(name = "VALUE", columnDefinition = "jsonb", nullable = false)
+     @ColumnTransformer(write = "?::jsonb")
+     private String value;
```

```
+
+     @Generated(GenerationTime.INSERT)
+     @Column(name = "CREATED_AT")
+     private Date createdAt;
+
+     @Generated(GenerationTime.ALWAYS)
+     @Column(name = "UPDATED_AT")
+     private Date updatedAt;
+ }
```

34

src/main/java/com/socgen/bigdata/catalog/repositories/attribute/AttributeRepository.java

```
@@ -0,0 +1,34 @@
+ package com.socgen.bigdata.catalog.repositories.attribute;
+
+ import com.socgen.bigdata.catalog.models.jpa.attribute.Attribute;
+ import java.util.List;
+ import java.util.Optional;
+ import org.springframework.data.jpa.repository.JpaRepository;
+ import org.springframework.stereotype.Repository;
+
+ @Repository
+ public interface AttributeRepository extends JpaRepository<Attribute, Long> {
+
+     List<Attribute> findAllByObjectType(String objectType);
+
+     List<Attribute> findAllByObjectTypeAndCategory(String objectType, String category);
+
+     /**
+      * Find by technicalName — primary lookup key for value push.
+      */
+     Optional<Attribute> findByObjectTypeAndCategoryAndTechnicalName(
+         String objectType,
+         String category,
+         String technicalName
+     );
+
+     /**
+      * Find by display name (NAME column) — fallback lookup for value push.
+      * Allows users to push values using either technicalNam    e or display name.
+      */
+     Optional<Attribute> findByObjectTypeAndCategoryAndName(
+         String objectType,
+         String category,
+         String name
+     );
+ }
```

17

...main/java/com/socgen/bigdata/catalog/repositories/attribute/AttributeValueRepository.ja…

```
@@ -0,0 +1,17 @@
+ package com.socgen.bigdata.catalog.repositories.attribute;
+
+ import com.socgen.bigdata.catalog.models.jpa.attribute.AttributeValue;
+ import java.util.Optional;
+ import org.springframework.data.jpa.repository.JpaRepository;
+ import org.springframework.stereotype.Repository;
+
+ @Repository
+ public interface AttributeValueRepository extends JpaRepository<AttributeValue, Long> {
+
+     /**
+      * Find the single row for a specific object instance.
+      * Unique on (OBJECT_TYPE, OBJECT_ID) — one row per object instance.
+      * Used for upsert on POST values.
+      */
+     Optional<AttributeValue> findByObjectTypeAndObjectId(String objectType, Long objectId);
+ }
```

51

src/main/java/com/socgen/bigdata/catalog/resources/v2/DatasetResource.java

```
import com.socgen.bigdata.catalog.models.dto.ProjectsDto;
import com.socgen.bigdata.catalog.models.dto.UpdatedDatasetBusinessDataDto;
import com.socgen.bigdata.catalog.models.dto.VersionsDto;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueRequestDto;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueResponseDto;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeViewResponseDto;
import com.socgen.bigdata.catalog.models.dto.datalocalization.DataLocalizationDto;
import com.socgen.bigdata.catalog.models.dto.dataset.AddedDatasetFromExcelFileDto;
import com.socgen.bigdata.catalog.models.dto.dataset.DatasetDataSourceAddInformationDto;
import com.socgen.bigdata.catalog.services.DataProjectService;
import com.socgen.bigdata.catalog.services.DxSearchService;
import com.socgen.bigdata.catalog.services.EntityService;
+ import com.socgen.bigdata.catalog.services.attribute.AttributeService;
import io.swagger.v3.oas.annotations.Operation;
```

```
36  40   import io.swagger.v3.oas.annotations.Parameter;
37  41   import io.swagger.v3.oas.annotations.media.ArraySchema;
44  48   import java.util.Map;
45  49   import java.util.Set;
46  50   import java.util.UUID;
    51 + import javax.validation.Valid;
47  52   import javax.validation.constraints.Max;
48  53   import lombok.AllArgsConstructor;
49  54   import lombok.Data;
104 109      @Autowired
105 110      private DataProjectService dataProjectService;
106 111
    112 +    @Autowired
    113 +    private AttributeService attributeService;
    114 +
107 115      @Operation(description = "Search all matching entities paginated")
108 116      @GetMapping(produces = APPLICATION_JSON_VALUE)
109 117      public ResponseEntity<List<EntityDto>> getEntities(
842 850
843 851          return ResponseEntity.ok(entityService.updateDataSources(id, version,
    action, dataSourceIdList));
844 852      }
    853 +
    854 +    @PostMapping(value = "/{id}/attributes", consumes =
    APPLICATION_JSON_VALUE, produces = APPLICATION_JSON_VALUE)
    855 +    @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2') and
    hasAuthority('api.sgdata-catalog.metadata.write')")
    856 +    @Operation(
    857 +        operationId = "upsertDatasetAttributeValues",
    858 +        summary = "Create or update attribute values for a dataset",
    859 +        description = "Pushes attribute values for a specific dataset."
    860 +    )
    861 +    @ApiResponses(value = {
    862 +        @ApiResponse(responseCode = "201", description = "Attribute values
    upserted successfully.",
    863 +            content = @Content(mediaType = APPLICATION_JSON_VALUE, schema =
    @Schema(implementation = AttributeValueResponseDto.class))),
    864 +        @ApiResponse(responseCode = "400", description = "Bad Request.",
    865 +            content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
    866 +        @ApiResponse(responseCode = "404", description = "Dataset not found.",
    867 +            content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
    868 +    })
    869 +    public ResponseEntity<AttributeValueResponseDto>
    upsertDatasetAttributeValues(
    870 +        @Parameter(description = "Dataset id", required = true, example = "1")
    871 +        @PathVariable(name = "id") Long datasetId,
    872 +        @Valid @RequestBody AttributeValueRequestDto requestDto
    873 +    ) {
    874 +        return ResponseEntity.status(HttpStatus.CREATED)
    875 +            .body(attributeService.upsertDatasetAttributeValues(datasetId,
    requestDto));
    876 +    }
```

```
+
+     @GetMapping(value = "/{id}/attributes", produces = APPLICATION_JSON_VALUE)
+     @Operation(
+         operationId = "getDatasetAttributeValues",
+         summary = "Get attribute values grouped by category for a dataset",
+         description = "Returns all stored attribute values for the given dataset grouped by category."
+     )
+     @ApiResponses(value = {
+         @ApiResponse(responseCode = "200", description = "Attribute values retrieved successfully.",
+             content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeViewResponseDto.class))),
+         @ApiResponse(responseCode = "404", description = "Dataset not found.",
+             content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
+     })
+     public ResponseEntity<AttributeViewResponseDto> getDatasetAttributeValues(
+         @Parameter(description = "Dataset id", required = true, example = "1")
+         @PathVariable(name = "id") Long datasetId
+     ) {
+         return ResponseEntity.ok(attributeService.getDatasetAttributeValues(datasetId));
+     }
}
```

368

src/main/java/com/socgen/bigdata/catalog/services/attribute/AttributeService.java

Load diff

Large diffs are not rendered by default.

64

...ava/com/socgen/bigdata/catalog/web/controller/v2/attribute/AttributeConfigController.ja…

```
@@ -0,0 +1,64 @@
+ package com.socgen.bigdata.catalog.web.controller.v2.attribute;
+
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeDefinitionRequestDto;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeDefinitionResponseDto;
+ import com.socgen.bigdata.catalog.services.attribute.AttributeService;
+ import io.swagger.v3.oas.annotations.Operation;
+ import io.swagger.v3.oas.annotations.Parameter;
```

```
+ import io.swagger.v3.oas.annotations.media.Content;
+ import io.swagger.v3.oas.annotations.media.Schema;
+ import io.swagger.v3.oas.annotations.responses.ApiResponse;
+ import io.swagger.v3.oas.annotations.responses.ApiResponses;
+ import io.swagger.v3.oas.annotations.tags.Tag;
+ import lombok.RequiredArgsConstructor;
+ import lombok.extern.slf4j.Slf4j;
+ import org.springframework.http.HttpStatus;
+ import org.springframework.http.ResponseEntity;
+ import org.springframework.security.access.prepost.PreAuthorize;
+ import org.springframework.validation.annotation.Validated;
+ import org.springframework.web.bind.annotation.*;
+
+ import javax.validation.Valid;
+
+ import static org.springframework.http.MediaType.APPLICATION_JSON_VALUE;
+ import static org.springframework.http.MediaType.APPLICATION_PROBLEM_JSON_VALUE;
+
+ @RestController
+ @RequestMapping(path = "/v2/api/entity-types", produces = APPLICATION_JSON_VALUE)
+ @Tag(name = "Attribute Configs", description = "Manage dynamic attribute configurations for Tables, DataSource and Dataset")
+ @Slf4j
+ @RequiredArgsConstructor
+ @Validated
+ @ApiResponses(value = {
+     @ApiResponse(responseCode = "401", description = "Unauthorized"),
+     @ApiResponse(responseCode = "403", description = "Forbidden"),
+     @ApiResponse(responseCode = "500", description = "Internal Server Error"),
+ })
+ public class AttributeConfigController {
+
+     private final AttributeService attributeService;
+
+     @PostMapping(value = "/{entityType}/attribute-configs", consumes = APPLICATION_JSON_VALUE)
+     @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2') and hasAuthority('api.sgdata-catalog.metadata.write')")
+     @Operation(
+         operationId = "upsertAttributeConfigs",
+         summary = "Create or update attribute configurations for an entity type",
+         description = "Upserts dynamic attribute configurations for a given entity type under a category. " +
+                       "If an attribute with the same technicalName already exists under the same entityType and category it will be updated. " +
+                       "entityType: tables | datasources | datasets."
+     )
+     @ApiResponses(value = {
```

```
+        @ApiResponse(responseCode = "201", description = "Attribute configs created or updated successfully.",
+            content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeDefinitionResponseDto.class))),
+        @ApiResponse(responseCode = "400", description = "Bad Request.",
+            content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
+    })
+    public ResponseEntity<AttributeDefinitionResponseDto> upsertAttributeConfigs(
+        @Parameter(description = "Entity type: Tables | DataSource | Dataset", required = true)
+        @PathVariable String entityType,
+        @Valid @RequestBody AttributeDefinitionRequestDto requestDto
+    ) {
+        return ResponseEntity.status(HttpStatus.CREATED)
+            .body(attributeService.upsertAttributeConfigs(entityType, requestDto));
+    }
+ }
```

57

...in/java/com/socgen/bigdata/catalog/web/controller/v2/datasource/DataSourceController.ja…

```
10  10  import com.socgen.bigdata.catalog.enums.datasource.DataSourceEnvironment;
11  11  import com.socgen.bigdata.catalog.enums.datasource.DataSourceStatus;
12  12  import com.socgen.bigdata.catalog.helpers.PaginationHelper;
    13 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueRequestDto;
    14 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueResponseDto;
    15 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeViewResponseDto;
13  16  import com.socgen.bigdata.catalog.models.dto.dam.FeederDto;
14  17  import com.socgen.bigdata.catalog.models.dto.dam.StorageAccountDto;
15  18  import com.socgen.bigdata.catalog.models.dto.datasource.AddDataSourceRequestDto;
23  26  import com.socgen.bigdata.catalog.models.dto.datasource.response.AddDataSourceResourceResponseDto;
24  27  import com.socgen.bigdata.catalog.models.dto.legacydatasource.DataSourceResourceUpdateFileResultDto;
25  28  import com.socgen.bigdata.catalog.repositories.datasource.DataSourceCriteria;
    29 + import com.socgen.bigdata.catalog.services.attribute.AttributeService;
26  30  import com.socgen.bigdata.catalog.services.dam.DataLakeAccessManagementService;
27  31  import com.socgen.bigdata.catalog.services.datasource.DataSourceService;
28  32  import com.socgen.bigdata.catalog.services.datasource.DataSourceTableService;
84  88      private final DataSourceService dataSourceService;
85  89      private final DataLakeAccessManagementService dataLakeAccessManagementService;
```

```
86  90      private final DataSourceTableService dataSourceTableService;
    91  +   private final AttributeService attributeService;
87  92
88  93      public DataSourceController(
89  94          DataSourceService dataSourceService,
90  95          DataSourceTableService dataSourceTableService,
91      -       DataLakeAccessManagementService dataLakeAccessManagementService
    96  +       DataLakeAccessManagementService dataLakeAccessManagementService,
    97  +       AttributeService attributeService
92  98      ) {
93  99          this.dataSourceService = dataSourceService;
94  100         this.dataSourceTableService = dataSourceTableService;
95  101         this.dataLakeAccessManagementService = dataLakeAccessManagementService;
    102 +       this.attributeService = attributeService;
96  103     }
97  104
98  105     @GetMapping(produces = APPLICATION_JSON_VALUE)
375 382         return PaginationHelper.responseEntityOk(
376 383         dataSourceTableService.getDataSourceTableDtoPageByDataSourceIdAndExistingOnlyAndPageable(
377 384                 id,
378     -               existingOnly,
    385 +               false, //change this line to be checked while PR Review
379 386                 pageable
380 387             )
381 388         );
784 791             throw new ResponseStatusException(HttpStatus.UNPROCESSABLE_ENTITY, message, e);
785 792         }
786 793     }
    794 +
    795 +   @PostMapping(value = "/{id}/attributes", consumes = APPLICATION_JSON_VALUE, produces = APPLICATION_JSON_VALUE)
    796 +   @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2') and hasAuthority('api.sgdata-catalog.metadata.write')")
    797 +   @Operation(
    798 +       operationId = "upsertDataSourceAttributeValues",
    799 +       summary = "Create or update attribute values for a data source",
    800 +       description = "Pushes attribute values for a specific data source. " +
    801 +                     "Accepts both technicalName and display name as key. " +
    802 +                     "Values are merged — existing keys not in the request are preserved."
    803 +   )
    804 +   @ApiResponses(value = {
    805 +       @ApiResponse(responseCode = "201", description = "Attribute values upserted successfully.",
    806 +           content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeValueResponseDto.class))),
    807 +       @ApiResponse(responseCode = "400", description = "Bad Request.",
    808 +           content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
```

```
809 +         @ApiResponse(responseCode = "404", description = "DataSource not found.",
810 +             content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
811 +     })
812 +     public ResponseEntity<AttributeValueResponseDto> upsertDataSourceAttributeValues(
813 +         @Parameter(description = "Data source id", required = true, example = "1")
814 +         @PathVariable(name = "id") Long datasourceId,
815 +         @Valid @RequestBody AttributeValueRequestDto requestDto
816 +     ) {
817 +         return ResponseEntity.status(HttpStatus.CREATED)
818 +         .body(attributeService.upsertDataSourceAttributeValues(datasourceId, requestDto));
819 +     }
820 +
821 +     @GetMapping(value = "/{id}/attributes", produces = APPLICATION_JSON_VALUE)
822 +     @Operation(
823 +         operationId = "getDataSourceAttributeValues",
824 +         summary = "Get attribute values grouped by category for a data source",
825 +         description = "Returns all stored attribute values for the given data source grouped by category."
826 +     )
827 +     @ApiResponses(value = {
828 +         @ApiResponse(responseCode = "200", description = "Attribute values retrieved successfully.",
829 +             content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeViewResponseDto.class))),
830 +         @ApiResponse(responseCode = "404", description = "DataSource not found.",
831 +             content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
832 +     })
833 +     public ResponseEntity<AttributeViewResponseDto> getDataSourceAttributeValues(
834 +         @Parameter(description = "Data source id", required = true, example = "1")
835 +         @PathVariable(name = "id") Long datasourceId
836 +     ) {
837 +         return ResponseEntity.ok(attributeService.getDataSourceAttributeValues(datasourceId));
838 +     }
839 +
787 840   }
```

58

...va/com/socgen/bigdata/catalog/web/controller/v2/datasource/DataSourceTableController.ja…

```
9   9   import com.socgen.bigdata.catalog.helpers.PaginationHelper;
```

```
10  10   import com.socgen.bigdata.catalog.models.dto.BulkBusinessDataResultDto;
11  11   import com.socgen.bigdata.catalog.models.dto.HiveDataSourceDto;
    12 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueRequestDto;
    13 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeValueResponseDto;
    14 + import com.socgen.bigdata.catalog.models.dto.attribute.AttributeViewResponseDto;
12  15   import com.socgen.bigdata.catalog.models.dto.datasource.DataSourceTableDetailsColumnDto;
13  16   import com.socgen.bigdata.catalog.models.dto.datasource.DataSourceTableDto;
14  17   import com.socgen.bigdata.catalog.models.dto.datasource.DataSourceTableWithColumnsDto;
15  18   import com.socgen.bigdata.catalog.models.jpa.datasource.DataSourceTable;
16  19   import com.socgen.bigdata.catalog.repositories.datasource.DataSourceTableCriteria;
    20 + import com.socgen.bigdata.catalog.services.attribute.AttributeService;
17  21   import com.socgen.bigdata.catalog.services.datasource.DataSourceTableService;
18  22   import com.socgen.bigdata.catalog.services.mappers.datasource.DataSourceTableMapper;
19  23   import io.swagger.v3.oas.annotations.Operation;
34  38   import org.springframework.data.domain.PageRequest;
35  39   import org.springframework.data.domain.Pageable;
36  40   import org.springframework.data.domain.Sort;
    41 + import org.springframework.http.HttpStatus;
37  42   import org.springframework.http.MediaType;
38  43   import org.springframework.http.ResponseEntity;
39  44   import org.springframework.security.access.prepost.PreAuthorize;
66  71
67  72       private final DataSourceTableService dataSourceTableService;
68  73
    74 +     private final AttributeService attributeService;
    75 +
69  76       public DataSourceTableController(
70  77           DataSourceTableMapper dataSourceTableMapper,
71     -         DataSourceTableService dataSourceTableService
    78 +         DataSourceTableService dataSourceTableService,
    79 +         AttributeService attributeService
72  80       ) {
73  81           this.dataSourceTableMapper = dataSourceTableMapper;
74  82           this.dataSourceTableService = dataSourceTableService;
    83 +         this.attributeService = attributeService;
75  84       }
76  85
77  86       @GetMapping(produces = APPLICATION_JSON_VALUE)
290 299          operationId = "bulkUpdateBusinessDataForTable",
291 300          summary = "Bulk update business data for a data source table",
292 301          description =
293    -             "Upload an Excel (.xlsx) file mapping ColumnName -> CollibraId to bulk update business data for the given table. " +
```

```
+            "Upload an Excel (.xlsx) file mapping ColumnName -> BusinessDataId to bulk update business data for the given table. " +
             "Optional dataSourceId can be provided to cross-check that the table belongs to the data source."
     )
     @ApiResponses(
     public ResponseEntity<BulkBusinessDataResultDto> bulkUpdateBusinessDataForTable(
         @Parameter(description = "Data source table id", required = true, example = "32716") @PathVariable(name = "id") Long tableId,
         @Parameter(description = "Data source id (optional, for verification)", example = "111") @RequestParam(name = "dataSourceId", required = false) Long dataSourceId,
-        @Parameter(description = "Excel file containing ColumnName and CollibraId (xlsx)", required = true) @RequestParam(name = "file") MultipartFile file
+        @Parameter(description = "Excel file containing ColumnName and BusinessDataId (xlsx)", required = true) @RequestParam(name = "file") MultipartFile file
     ) {
         log.info(
             "[Bulk business data upload] Started (tableId: {}, dataSourceId: {}, filename: {})...",
         return ResponseEntity.ok(result);
     }

+    @PostMapping(value = "/{id}/attributes", consumes = APPLICATION_JSON_VALUE, produces = APPLICATION_JSON_VALUE)
+    @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2') and hasAuthority('api.sgdata-catalog.metadata.write')")
+    @Operation(
+        operationId = "upsertTableAttributeValues",
+        summary = "Create or update attribute values for a data source table",
+        description = "Pushes attribute values for a specific data source table. "
+    )
+    @ApiResponses(value = {
+        @ApiResponse(responseCode = "201", description = "Attribute values upserted successfully.",
+            content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeValueResponseDto.class))),
+        @ApiResponse(responseCode = "400", description = "Bad Request.",
+            content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
+        @ApiResponse(responseCode = "404", description = "Table not found.",
+            content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
+    })
+    public ResponseEntity<AttributeValueResponseDto> upsertTableAttributeValues(
+        @Parameter(description = "Data source table id", required = true, example = "1")
+        @PathVariable(name = "id") Long tableId,
+        @Valid @RequestBody AttributeValueRequestDto requestDto
```

```
+     ) {
+         return ResponseEntity.status(HttpStatus.CREATED)
+             .body(attributeService.upsertTableAttributeValues(tableId, requestDto));
+     }
+
+     @GetMapping(value = "/{id}/attributes", produces = APPLICATION_JSON_VALUE)
+     @Operation(
+         operationId = "getTableAttributeValues",
+         summary = "Get attribute values grouped by category for a data source table",
+         description = "Returns all stored attribute values for the given table grouped by category. "
+
+     )
+     @ApiResponses(value = {
+         @ApiResponse(responseCode = "200", description = "Attribute values retrieved successfully.",
+             content = @Content(mediaType = APPLICATION_JSON_VALUE, schema = @Schema(implementation = AttributeViewResponseDto.class))),
+         @ApiResponse(responseCode = "404", description = "Table not found.",
+             content = @Content(mediaType = APPLICATION_PROBLEM_JSON_VALUE)),
+     })
+     public ResponseEntity<AttributeViewResponseDto> getTableAttributeValues(
+         @Parameter(description = "Data source table id", required = true, example = "1")
+         @PathVariable(name = "id") Long tableId
+     ) {
+         return ResponseEntity.ok(attributeService.getTableAttributeValues(tableId));
+     }
}
```

4 src/main/resources/db/changelog/db.changelog-master.yaml

```
  - include:
      file: db/changelog/db.changelog.fix-audit-type-snowflake.yaml
  - include:
-     file: db/changelog/db.changelog.add-exist-in-collibra-column.yaml
+     file: db/changelog/db.changelog.add-exist-in-collibra-column.yaml
+ - include:
+     file: db/changelog/db.changelog.add-attribute-tables.yaml
```

123 src/main/resources/db/changelog/db.changelog.add-attribute-tables.yaml

```
@@ -0,0 +1,123 @@
+ databaseChangeLog:
+   - changeSet:
```

```yaml
+      id: create-attribute
+      author: viknesh vallinayagam
+      changes:
+        - createTable:
+            tableName: ATTRIBUTE
+            columns:
+              - column:
+                  name: ID
+                  type: BIGINT
+                  autoIncrement: true
+                  constraints:
+                    nullable: false
+              - column:
+                  name: NAME
+                  type: VARCHAR(255)
+                  constraints:
+                    nullable: false
+              - column:
+                  name: TECHNICAL_NAME
+                  type: VARCHAR(255)
+                  constraints:
+                    nullable: false
+              - column:
+                  name: DATA_TYPE
+                  type: VARCHAR(50)
+                  constraints:
+                    nullable: false
+              - column:
+                  name: ENTITY_TYPE
+                  type: VARCHAR(50)
+                  constraints:
+                    nullable: false
+              - column:
+                  name: CATEGORY
+                  type: VARCHAR(255)
+                  constraints:
+                    nullable: false
+              - column:
+                  name: READ_ONLY
+                  type: BOOLEAN
+                  defaultValueBoolean: false
+                  constraints:
+                    nullable: false
+              - column:
+                  name: PROPERTIES
+                  type: JSONB
+              - column:
+                  name: CREATED_AT
+                  type: TIMESTAMPTZ
+                  defaultValueComputed: NOW()
+                  constraints:
+                    nullable: false
```

```
+                - column:
+                    name: UPDATED_AT
+                    type: TIMESTAMPTZ
+                    defaultValueComputed: NOW()
+                    constraints:
+                      nullable: false
+          - addPrimaryKey:
+              tableName: ATTRIBUTE
+              columnNames: ID
+              constraintName: PK_ATTRIBUTE
+          - addUniqueConstraint:
+              tableName: ATTRIBUTE
+              columnNames: ENTITY_TYPE, CATEGORY, TECHNICAL_NAME
+              constraintName: UQ_ATTRIBUTE_ENTITY_TYPE_CATEGORY_TECHNICAL_NAME
+        rollback:
+          - dropTable:
+              tableName: ATTRIBUTE
+
+    - changeSet:
+        id: create-attribute-value
+        author: viknesh vallinayagam
+        changes:
+          - createTable:
+              tableName: ATTRIBUTE_VALUE
+              columns:
+                - column:
+                    name: ID
+                    type: BIGINT
+                    autoIncrement: true
+                    constraints:
+                      nullable: false
+                - column:
+                    name: ENTITY_TYPE
+                    type: VARCHAR(50)
+                    constraints:
+                      nullable: false
+                - column:
+                    name: ENTITY_ID
+                    type: BIGINT
+                    constraints:
+                      nullable: false
+                - column:
+                    name: VALUE
+                    type: JSONB
+                    constraints:
+                      nullable: false
+                - column:
+                    name: CREATED_AT
+                    type: TIMESTAMPTZ
+                    defaultValueComputed: NOW()
+                    constraints:
+                      nullable: false
```

```
+                - column:
+                    name: UPDATED_AT
+                    type: TIMESTAMPTZ
+                    defaultValueComputed: NOW()
+                    constraints:
+                      nullable: false
+          - addPrimaryKey:
+              tableName: ATTRIBUTE_VALUE
+              columnNames: ID
+              constraintName: PK_ATTRIBUTE_VALUE
+          - addUniqueConstraint:
+              tableName: ATTRIBUTE_VALUE
+              columnNames: ENTITY_TYPE, ENTITY_ID
+              constraintName: UQ_ATTRIBUTE_VALUE_ENTITY_TYPE_ENTITY_ID
+        rollback:
+          - dropTable:
+              tableName: ATTRIBUTE_VALUE
```

72

src/test/java/com/socgen/bigdata/catalog/models/jpa/attribute/AttributeDataTypeTest.java

```
@@ -0,0 +1,72 @@
+ package com.socgen.bigdata.catalog.models.jpa.attribute;
+
+ import org.junit.jupiter.api.Test;
+ import org.junit.jupiter.params.ParameterizedTest;
+ import org.junit.jupiter.params.provider.CsvSource;
+ import org.junit.jupiter.params.provider.NullSource;
+
+ import static org.junit.jupiter.api.Assertions.assertEquals;
+ import static org.junit.jupiter.api.Assertions.assertThrows;
+
+
+ class AttributeDataTypeTest {
+
+
+     @ParameterizedTest(name = "fromString({0}) -> {1}")
+     @CsvSource({
+         "string,  STRING",
+         "STRING,  STRING",
+         "String,  STRING",
+         "integer, INTEGER",
+         "INTEGER, INTEGER",
+         "float,   FLOAT",
+         "FLOAT,   FLOAT",
+         "boolean, BOOLEAN",
+         "BOOLEAN, BOOLEAN",
+     })
+     void
  Given_ValidDataTypeString_When_FromString_Then_ReturnsCorrectEnum(String
  input, AttributeDataType expected) {
```

```
+        assertEquals(expected, AttributeDataType.fromString(input.trim()));
+    }
+
+    @ParameterizedTest
+    @NullSource
+    void Given_NullDataType_When_FromString_Then_ThrowsIllegalArgumentException(String input) {
+        IllegalArgumentException ex = assertThrows(
+            IllegalArgumentException.class,
+            () -> AttributeDataType.fromString(input)
+        );
+        assertEquals("dataType must not be null.", ex.getMessage());
+    }
+
+    @Test
+    void Given_UnknownDataType_When_FromString_Then_ThrowsIllegalArgumentExceptionWithHelpfulMessage() {
+        IllegalArgumentException ex = assertThrows(
+            IllegalArgumentException.class,
+            () -> AttributeDataType.fromString("text")
+        );
+        String msg = ex.getMessage();
+        org.junit.jupiter.api.Assertions.assertTrue(msg.contains("text"), "Message should mention the invalid value");
+        org.junit.jupiter.api.Assertions.assertTrue(
+            msg.toLowerCase().contains("string") || msg.toLowerCase().contains("allowed"),
+            "Message should hint at allowed values"
+        );
+    }
+
+    // ---------------------------------------------------------------- toJsonSchemaType
+
+    @ParameterizedTest(name = "{0}.toJsonSchemaType() -> \"{1}\"")
+    @CsvSource({
+        "STRING,  string",
+        "INTEGER, integer",
+        "FLOAT,   number",
+        "BOOLEAN, boolean",
+        "ARRAY,   array",
+        "OBJECT,  object",
+    })
+    void Given_AttributeDataTypeEnum_When_ToJsonSchemaType_Then_ReturnsExpectedJsonSchemaString(
+        AttributeDataType dataType, String expectedJsonType
+    ) {
+        assertEquals(expectedJsonType.trim(), dataType.toJsonSchemaType());
+    }
```

```
+ }
+
```

817

src/test/java/com/socgen/bigdata/catalog/services/attribute/AttributeServiceTest.java

Load diff

Large diffs are not rendered by default.

391

...a/com/socgen/bigdata/catalog/web/controller/v2/attribute/AttributeConfigControllerIT.ja…

```
@@ -0,0 +1,391 @@
+ package com.socgen.bigdata.catalog.web.controller.v2.attribute;
+
+ import com.fasterxml.jackson.databind.ObjectMapper;
+ import com.socgen.apibank.security.sgconnect.test.support.WithSgConnectUser;
+ import com.socgen.bigdata.catalog.config.DataCatalogIntegrationTest;
+ import com.socgen.bigdata.catalog.exceptions.InvalidInputException;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeDefinitionRequestDto;
+ import com.socgen.bigdata.catalog.models.dto.attribute.AttributeDefinitionResponseDto;
+ import com.socgen.bigdata.catalog.services.attribute.AttributeService;
+ import org.junit.jupiter.api.Test;
+ import org.springframework.beans.factory.annotation.Autowired;
+ import org.springframework.boot.test.mock.mockito.MockBean;
+ import org.springframework.http.MediaType;
+ import org.springframework.test.web.servlet.MockMvc;
+
+ import java.util.Collections;
+ import java.util.List;
+ import java.util.Map;
+
+ import static org.mockito.ArgumentMatchers.any;
+ import static org.mockito.ArgumentMatchers.eq;
+ import static org.mockito.BDDMockito.given;
+ import static org.mockito.Mockito.*;
+ import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
+ import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
+ import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```
+
+ @DataCatalogIntegrationTest
+ class AttributeConfigControllerIT {
+
+     private static final String BASE_URI = "/v2/api/entity-types";
+     private static final String CORRECT_SCOPE = "api.sgdata-catalog.attributes";
+
+     @Autowired
+     private MockMvc mockMvc;
+
+     @Autowired
+     private ObjectMapper objectMapper;
+
+     @MockBean
+     private AttributeService attributeService;
+
+
+     private String toJson(Object obj) throws Exception {
+         return objectMapper.writeValueAsString(obj);
+     }
+
+     private AttributeDefinitionRequestDto singleAttributeRequest() {
+         return AttributeDefinitionRequestDto.builder()
+             .category("Technical")
+             .attributes(List.of(
+                 AttributeDefinitionRequestDto.AttributeItemDto.builder()
+                     .name("Number Of Files")
+                     .technicalName("numberOfFiles")
+                     .dataType("integer")
+                     .readOnly(false)
+                     .build()
+             ))
+             .build();
+     }
+
+     private AttributeDefinitionResponseDto singleAttributeResponse(String entityType) {
+         return AttributeDefinitionResponseDto.builder()
+             .objectType(entityType)
+             .category("Technical")
+             .createdOrUpdated(List.of(
+                 AttributeDefinitionResponseDto.CreatedAttributeDto.builder()
+                     .attributeDefId(1L)
+                     .name("Number Of Files")
+                     .technicalName("numberOfFiles")
+                     .build()
+             ))
+             .build();
+     }
+
+
```

```
+     @Test
+     @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+     void Given_ValidRequest_When_UpsertAttributeConfigs_Then_Returns201WithBody() throws Exception {
+         // given
+         String entityType = "Tables";
+         AttributeDefinitionRequestDto request = singleAttributeRequest();
+         AttributeDefinitionResponseDto response = singleAttributeResponse(entityType);
+
+         given(attributeService.upsertAttributeConfigs(eq(entityType), any(AttributeDefinitionRequestDto.class)))
+             .willReturn(response);
+
+         // when & then
+         mockMvc.perform(
+                 post(BASE_URI + "/{entityType}/attribute-configs", entityType)
+                     .contentType(MediaType.APPLICATION_JSON)
+                     .content(toJson(request))
+             )
+             .andExpect(status().isCreated())
+             .andExpect(jsonPath("$.objectType").value("Tables"))
+             .andExpect(jsonPath("$.category").value("Technical"))
+             .andExpect(jsonPath("$.createdOrUpdated").isArray())
+             .andExpect(jsonPath("$.createdOrUpdated[0].attributeDefId").value(1))
+             .andExpect(jsonPath("$.createdOrUpdated[0].name").value("Number Of Files"))
+             .andExpect(jsonPath("$.createdOrUpdated[0].technicalName").value("numberOfFiles"));
+
+         verify(attributeService, times(1))
+             .upsertAttributeConfigs(eq(entityType), any(AttributeDefinitionRequestDto.class));
+     }
+
+
+     @Test
+     @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+     void Given_MultipleAttributes_When_UpsertAttributeConfigs_Then_Returns201WithAllCreatedEntries() throws Exception {
+         // given
+         String entityType = "DataSource";
+
+         AttributeDefinitionRequestDto request = AttributeDefinitionRequestDto.builder()
+             .category("Business")
+             .attributes(List.of(
+                 AttributeDefinitionRequestDto.AttributeItemDto.builder()
```

```
+                    .name("Owner")
+                    .technicalName("owner")
+                    .dataType("string")
+                    .build(),
+                AttributeDefinitionRequestDto.AttributeItemDto.builder()
+                    .name("Is Active")
+                    .technicalName("isActive")
+                    .dataType("boolean")
+                    .readOnly(true)
+                    .build()
+            ))
+            .build();
+
+        AttributeDefinitionResponseDto response = AttributeDefinitionResponseDto.builder()
+            .objectType(entityType)
+            .category("Business")
+            .createdOrUpdated(List.of(
+                AttributeDefinitionResponseDto.CreatedAttributeDto.builder()
+ .attributeDefId(10L).name("Owner").technicalName("owner").build(),
+                AttributeDefinitionResponseDto.CreatedAttributeDto.builder()
+                    .attributeDefId(11L).name("Is Active").technicalName("isActive").build()
+            ))
+            .build();
+
+        given(attributeService.upsertAttributeConfigs(eq(entityType), any())).willReturn(response);
+
+        // when & then
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", entityType)
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(toJson(request))
+            )
+            .andExpect(status().isCreated())
+            .andExpect(jsonPath("$.objectType").value("DataSource"))
+            .andExpect(jsonPath("$.createdOrUpdated").isArray())
+            .andExpect(jsonPath("$.createdOrUpdated.length()").value(2))
+ .andExpect(jsonPath("$.createdOrUpdated[0].technicalName").value("owner"))
+ .andExpect(jsonPath("$.createdOrUpdated[1].technicalName").value("isActive"));
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_BlankCategory_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        String body = """
```

```
+            {
+              "category": "",
+              "attributes": [
+                { "name": "Count", "technicalName": "count", "dataType": "integer" }
+              ]
+            }
+            """;
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_EmptyAttributesList_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        String body = """
+            {
+              "category": "Technical",
+              "attributes": []
+            }
+            """;
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_BlankAttributeName_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        String body = """
+            {
+              "category": "Technical",
+              "attributes": [
```

```
+                { "name": "", "technicalName": "count", "dataType": "integer" }
+              ]
+            }
+            """;
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_BlankTechnicalName_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        String body = """
+            {
+              "category": "Technical",
+              "attributes": [
+                { "name": "Count", "technicalName": "", "dataType": "integer" }
+              ]
+            }
+            """;
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_BlankDataType_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        String body = """
+            {
+              "category": "Technical",
+              "attributes": [
+                { "name": "Count", "technicalName": "count", "dataType": "" }
```

```
+                ]
+            }
+            """;
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_MissingRequestBody_When_UpsertAttributeConfigs_Then_Returns400() throws Exception {
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    // no body
+            )
+            .andExpect(status().isBadRequest());
+
+        verifyNoInteractions(attributeService);
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { "api.sgdata-catalog.v2" })  // wrong scope
+    void Given_WrongScope_When_UpsertAttributeConfigs_Then_Returns403() throws Exception {
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(toJson(singleAttributeRequest()))
+            )
+            .andExpect(status().isForbidden());
+
+        verify(attributeService, never()).upsertAttributeConfigs(any(), any());
+    }
+
+
+    @Test
+    void Given_NoAuthentication_When_UpsertAttributeConfigs_Then_Returns401() throws Exception {
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", "Tables")
```

```
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(toJson(singleAttributeRequest()))
+            )
+            .andExpect(status().isUnauthorized());
+
+        verify(attributeService, never()).upsertAttributeConfigs(any(), any());
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_UnknownEntityType_When_UpsertAttributeConfigs_Then_Returns422() throws Exception {
+        String entityType = "UnknownType";
+
+        given(attributeService.upsertAttributeConfigs(eq(entityType), any()))
+            .willThrow(new InvalidInputException(
+                "Invalid entityType 'UnknownType'",
+                Map.of("entityType", Collections.singletonList("Allowed values: [Tables, DataSource, Dataset]"))
+            ));
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", entityType)
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(toJson(singleAttributeRequest()))
+            )
+            .andExpect(status().isUnprocessableEntity());
+
+        verify(attributeService, times(1)).upsertAttributeConfigs(eq(entityType), any());
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_ReadOnlyOmitted_When_UpsertAttributeConfigs_Then_Returns201() throws Exception {
+        String entityType = "Dataset";
+
+        // readOnly not included in JSON
+        String body = """
+            {
+              "category": "Technical",
+              "attributes": [
+                { "name": "Size", "technicalName": "size", "dataType": "integer" }
+              ]
+            }
+            """;
+
```

```
+        AttributeDefinitionResponseDto response = AttributeDefinitionResponseDto.builder()
+            .objectType(entityType)
+            .category("Technical")
+            .createdOrUpdated(List.of(
+                AttributeDefinitionResponseDto.CreatedAttributeDto.builder()
+                .attributeDefId(2L).name("Size").technicalName("size").build()
+            ))
+            .build();
+
+        given(attributeService.upsertAttributeConfigs(eq(entityType), any())).willReturn(response);
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", entityType)
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(body)
+            )
+            .andExpect(status().isCreated())
+            .andExpect(jsonPath("$.objectType").value("Dataset"))
+            .andExpect(jsonPath("$.createdOrUpdated[0].attributeDefId").value(2))
+            .andExpect(jsonPath("$.createdOrUpdated[0].technicalName").value("size"));
+
+        verify(attributeService, times(1)).upsertAttributeConfigs(eq(entityType), any());
+    }
+
+
+    @Test
+    @WithSgConnectUser(scopes = { CORRECT_SCOPE })
+    void Given_DatasetEntityType_When_UpsertAttributeConfigs_Then_Returns201() throws Exception {
+        String entityType = "Dataset";
+        AttributeDefinitionResponseDto response = singleAttributeResponse(entityType);
+
+        given(attributeService.upsertAttributeConfigs(eq(entityType), any())).willReturn(response);
+
+        mockMvc.perform(
+                post(BASE_URI + "/{entityType}/attribute-configs", entityType)
+                    .contentType(MediaType.APPLICATION_JSON)
+                    .content(toJson(singleAttributeRequest()))
+            )
+            .andExpect(status().isCreated())
+            .andExpect(jsonPath("$.objectType").value("Dataset"));
+    }
+ }
+
```